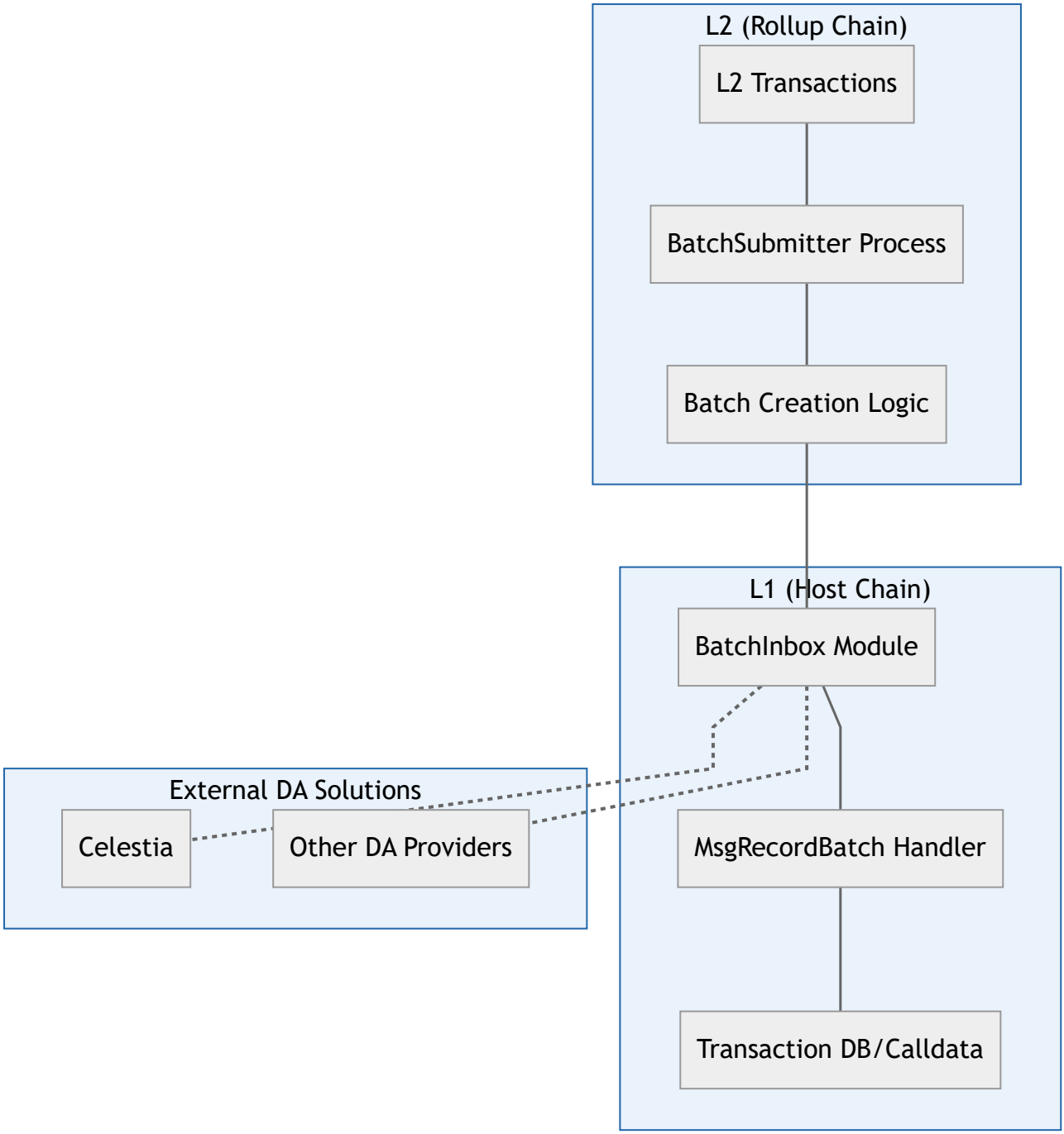
08. BatchSubmitter 시스템 분석

개요

BatchSubmitter는 OPinit 아키텍처에서 L2 트랜잭션 데이터를 L1의 BatchInbox 모듈에 제출하는 핵심 백그라운드 프로세스입니다. 이 시스템은 데이터 가용성(DA) 레이어의 역할을 수행하며, 롤업 체인의 결정론적 재구성을 가능하게 하는 중요한 컴포넌트입니다.

BatchSubmitter 아키텍처

1. 시스템 구성 요소



2. BatchSubmitter 프로세스 흐름

BatchSubmitter는 다음과 같은 단계로 동작합니다:

1. **트랜잭션 수집**: L2에서 발생한 트랜잭션들을 배치 단위로 수집
2. **배치 생성**: 수집된 트랜잭션들을 하나의 배치로 압축 및 직렬화
3. **L1 제출**: `MsgRecordBatch` 를 통해 L1 BatchInbox에 배치 데이터 제출
4. **가스 최적화**: Calldata 저장을 통한 비용 효율적 데이터 저장

BatchInbox 모듈 분석

1. MsgRecordBatch 구조

BatchInbox 모듈의 핵심은 `MsgRecordBatch` 메시지입니다:

```
// MsgRecordBatch는 트랜잭션 인덱싱을 위한 no-op 메시지
type MsgRecordBatch struct {
Submitter  string  `protobuf:"bytes,1,opt,name=submitter,proto3" json:"submitter
BridgeId  uint64  `protobuf:"varint,2,opt,name=bridge_id,json=bridgeId,proto3" j
BatchBytes []byte  `protobuf:"bytes,3,opt,name=batch_bytes,json=batchBytes,proto
}
```

필드 설명

- **Submitter**: 배치를 제출하는 BatchSubmitter의 주소
- **BridgeId**: 해당 배치가 속한 브리지의 고유 ID
- **BatchBytes**: 실제 L2 트랜잭션 배치 데이터 (압축된 바이트 배열)

2. 메시지 검증 로직

```
func (msg MsgRecordBatch) Validate(ac address.Codec) error {
// Submitter 주소 유효성 검사
if  _, err := ac.StringToBytes(msg.Submitter);  err != nil {
return  err
}
```

```go
// BridgeId 유효성 검사
if msg.BridgeId == 0 {
return ErrInvalidBridgeId
}

// BatchBytes 비어있지 않은지 확인
if len(msg.BatchBytes) == 0 {
return ErrEmptyBatchBytes
}

return nil
}
```

3. RecordBatch 핸들러 구현

```go
func (ms MsgServer) RecordBatch(ctx context.Context, req *types.MsgRecordBatch)
// 메시지 유효성 검사
if err := req.Validate(ms.authKeeper.AddressCodec()); err != nil {
return nil, err
}

// 배치 기록 이벤트 발생 (실제 상태 변경 없음)
```

```
sdk.UnwrapSDKContext(ctx).EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeRecordBatch,
sdk.NewAttribute(types.AttributeKeySubmitter, req.Submitter),
),
)

return &types.MsgRecordBatchResponse{}, nil
}
```

핵심 특징: `RecordBatch` 핸들러는 실제로 상태를 변경하지 않는 no-op 함수입니다. 이는 가스 비용 최적화를 위한 설계입니다.

데이터 가용성(DA) 레이어 역할

1. DA 레이어의 핵심 개념

BatchInbox 모듈은 OPinit에서 데이터 가용성 레이어 역할을 수행합니다:

```
// BridgeConfig에서 BatchInfo 설정
type BatchInfo struct {
Submitter  string  `json:"submitter,omitempty"`
ChainType BatchInfo_ChainType `json:"chain_type,omitempty"`
}
```

```
// 지원되는 체인 타입
const (
BatchInfo_UNSPECIFIED BatchInfo_ChainType = 0
BatchInfo_INITIA BatchInfo_ChainType = 1  // Initia 체인
BatchInfo_CELESTIA BatchInfo_ChainType = 2  // Celestia DA
)
```

2. 교체 가능한 DA 솔루션

BatchInbox는 다양한 DA 솔루션으로 교체 가능하도록 설계되었습니다:

Celestia 통합 예시

```
// BridgeConfig 생성 시 Celestia 설정
config := BridgeConfig{
BatchInfo: BatchInfo{
Submitter: "celestia_submitter_address",
ChainType: BatchInfo_CELESTIA,
},
// 기타 설정...
}
```

3. 결정론적 체인 재구성

DA 레이어의 핵심 기능은 롤업 체인의 결정론적 재구성입니다:

1. **데이터 저장**: 모든 L2 트랜잭션 데이터가 DA 레이어에 저장
2. **순서 보장**: 배치 제출 순서를 통한 트랜잭션 순서 보장
3. **무결성 검증**: 저장된 데이터를 통한 전체 체인 상태 재구성 가능

가스 비용 최적화 메커니즘

1. Calldata 저장 방식

BatchInbox의 핵심 최적화는 데이터를 상태(state)가 아닌 트랜잭션 데이터(calldata)에 저장하는 것입니다:

```
// MsgRecordBatch는 실제로 상태를 변경하지 않음
// 대신 트랜잭션의 calldata에 BatchBytes가 저장됨
func (ms MsgServer) RecordBatch(ctx context.Context, req *types.MsgRecordBatch)
// 상태 변경 없이 이벤트만 발생
// req.BatchBytes는 트랜잭션 calldata에 저장됨
sdk.UnwrapSDKContext(ctx).EventManager().EmitEvent(
sdk.NewEvent(types.EventTypeRecordBatch, ...)
)
return &types.MsgRecordBatchResponse{}, nil
}
```

2. 가스 비용 비교

저장 방식	가스 비용	데이터 접근성	영구성

| State Storage | 높음 (20,000 gas/32bytes) | 쉬움 | 높음 |

| Calldata Storage | 낮음 (16 gas/byte) | 보통 | 높음 |

| Event Log | 매우 낮음 (8 gas/byte) | 어려움 | 보통 |

BatchInbox는 Calldata 저장 방식을 사용하여 비용 효율성과 데이터 접근성의 균형을 맞춥니다.

3. 배치 크기 최적화

```go
const (
MaxDataLength = 1024 * 10  // 10KB 최대 데이터 크기
MaxMetadataLength = 1024 * 5  // 5KB 최대 메타데이터 크기
)

// 배치 크기 검증
func (msg MsgRecordBatch) Validate(ac address.Codec) error {
if len(msg.BatchBytes) == 0 {
return ErrEmptyBatchBytes
}
// 실제 구현에서는 최대 크기 제한도 확인
if len(msg.BatchBytes) > MaxBatchSize {
return ErrBatchTooLarge
}
return nil
}
```

BatchSubmitter 설정 및 관리

1. 브리지 생성 시 BatchSubmitter 설정

```
// CreateBridge 메시지에서 BatchInfo 설정
func (ms MsgServer) CreateBridge(ctx context.Context, req *types.MsgCreateBridge
// BatchInfo 검증
if  req.Config.BatchInfo.ChainType == BatchInfo_UNSPECIFIED ||
req.Config.BatchInfo.Submitter == "" {
return  nil, ErrEmptyBatchInfo
}
// 브리지 생성 이벤트에 BatchSubmitter 정보 포함
sdk.UnwrapSDKContext(ctx).EventManager().EmitEvent(sdk.NewEvent(
types.EventTypeCreateBridge,
sdk.NewAttribute(types.AttributeKeyBatchSubmitter, req.Config.BatchInfo.Submitte
sdk.NewAttribute(types.AttributeKeyBatchChainType, req.Config.BatchInfo.ChainTyp
// 기타 속성들...
))
return &types.MsgCreateBridgeResponse{BridgeId: bridgeId}, nil
}
```

2. BatchInfo 업데이트

```
// BatchInfo 업데이트 메시지
func (ms MsgServer) UpdateBatchInfo(ctx context.Context, req *types.MsgUpdateBat
// 권한 확인 (거버넌스 또는 현재 proposer만 가능)
if ms.authority != req.Authority && config.Proposer != req.Authority {
return nil, govtypes.ErrInvalidSigner.Wrapf("invalid authority")
}
// 새로운 BatchInfo 설정
config.BatchInfo = req.NewBatchInfo
// 브리지 설정 업데이트
if err := ms.SetBridgeConfig(ctx, bridgeId, config); err != nil {
return nil, err
}
// 업데이트 이벤트 발생
sdk.UnwrapSDKContext(ctx).EventManager().EmitEvent(sdk.NewEvent(
types.EventTypeUpdateBatchInfo,
sdk.NewAttribute(types.AttributeKeyBatchSubmitter, req.NewBatchInfo.Submitter),
sdk.NewAttribute(types.AttributeKeyBatchChainType, req.NewBatchInfo.ChainType.St
))
return &types.MsgUpdateBatchInfoResponse{}, nil
}
```

트랜잭션 배치 생성 프로세스

1. 배치 생성 알고리즘

BatchSubmitter는 다음과 같은 알고리즘으로 배치를 생성합니다:

```
// 의사 코드: BatchSubmitter 로직
type BatchSubmitter struct {
bridgeId  uint64
submitter  string
chainType BatchInfo_ChainType
batchSize  int
interval time.Duration
}

func (bs *BatchSubmitter) CreateBatch(transactions []L2Transaction) []byte {
// 1. 트랜잭션 직렬화
serializedTxs := make([][]byte, len(transactions))
for  i, tx := range  transactions {
serializedTxs[i] = tx.Serialize()
}
// 2. 배치 데이터 구성
batchData := BatchData{
Transactions: serializedTxs,
Timestamp: time.Now(),
BlockHeight: getCurrentL2Height(),
```

```go
}

// 3. 압축 (선택적)

compressedData := compress(batchData.Serialize())

return  compressedData

}


func (bs *BatchSubmitter) SubmitBatch(batchBytes []byte) error {

// MsgRecordBatch 생성

msg := &MsgRecordBatch{

Submitter: bs.submitter,

BridgeId: bs.bridgeId,

BatchBytes: batchBytes,

}

// L1에 트랜잭션 제출

return  bs.l1Client.SubmitTransaction(msg)

}
```

2. 배치 제출 주기 관리

```go
// BridgeConfig에서 제출 간격 설정

type BridgeConfig struct {

SubmissionInterval time.Duration // 배치 제출 간격

// 기타 필드들...
```

```go
}

// 주기적 배치 제출 로직
func (bs *BatchSubmitter) StartBatchSubmission() {
ticker := time.NewTicker(bs.config.SubmissionInterval)
defer ticker.Stop()
for {
select {
case <-ticker.C:
// 대기 중인 트랜잭션들을 배치로 생성
if pendingTxs := bs.getPendingTransactions(); len(pendingTxs) > 0 {
batchBytes := bs.CreateBatch(pendingTxs)
if err := bs.SubmitBatch(batchBytes); err != nil {
log.Error("Failed to submit batch", "error", err)
}
}
case <-bs.stopChan:
return
}
}
}
```

보안 고려사항

1. BatchSubmitter 권한 관리

```
// BatchSubmitter 권한 검증
func (ms MsgServer) RecordBatch(ctx context.Context, req *types.MsgRecordBatch)
// 브리지 설정 조회
bridgeConfig, err := ms.GetBridgeConfig(ctx, req.BridgeId)
if err != nil {
return nil, err
}
// 제출자 권한 확인
if req.Submitter != bridgeConfig.BatchInfo.Submitter {
return nil, sdkerrors.ErrUnauthorized.Wrap("invalid batch submitter")
}
// 나머지 로직...
}
```

2. 배치 데이터 무결성

```
// 배치 데이터 해시 검증 (선택적 구현)
type BatchHeader struct {
PrevBatchHash [32]byte
TxRoot [32]byte
Timestamp uint64
```

```
BatchSize  uint32
}


func (bs *BatchSubmitter) CreateSecureBatch(transactions []L2Transaction) []byte
// 트랜잭션 머클 루트 계산
txRoot := calculateMerkleRoot(transactions)
// 배치 헤더 생성
header := BatchHeader{
PrevBatchHash: bs.lastBatchHash,
TxRoot: txRoot,
Timestamp: uint64(time.Now().Unix()),
BatchSize: uint32(len(transactions)),
}
// 배치 데이터 구성
batchData := append(header.Serialize(), serializeTransactions(transactions)...)
// 현재 배치 해시 업데이트
bs.lastBatchHash = sha256.Sum256(batchData)
return  batchData
}
```

모니터링 및 디버깅

1. 배치 제출 이벤트

```
// RecordBatch 이벤트 구조
const (
EventTypeRecordBatch = "record_batch"
AttributeKeySubmitter = "submitter"
AttributeKeyBridgeId = "bridge_id"
AttributeKeyBatchSize = "batch_size"
AttributeKeyTimestamp = "timestamp"
)


// 확장된 이벤트 발생
sdk.UnwrapSDKContext(ctx).EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeRecordBatch,
sdk.NewAttribute(types.AttributeKeySubmitter, req.Submitter),
sdk.NewAttribute(types.AttributeKeyBridgeId, strconv.FormatUint(req.BridgeId, 10
sdk.NewAttribute(types.AttributeKeyBatchSize, strconv.Itoa(len(req.BatchBytes)))
sdk.NewAttribute(types.AttributeKeyTimestamp, strconv.FormatInt(time.Now().Unix(
),
)
```

2. 배치 제출 메트릭스

```go
// BatchSubmitter 성능 메트릭스
type BatchMetrics struct {
TotalBatchesSubmitted  uint64
TotalBytesSubmitted  uint64
AverageBatchSize  float64
LastSubmissionTime time.Time
FailedSubmissions  uint64
}

func (bs *BatchSubmitter) UpdateMetrics(batchSize  int, success  bool) {
bs.metrics.TotalBatchesSubmitted++
if  success {
bs.metrics.TotalBytesSubmitted += uint64(batchSize)
bs.metrics.AverageBatchSize = float64(bs.metrics.TotalBytesSubmitted) / float64(
bs.metrics.LastSubmissionTime = time.Now()
} else {
bs.metrics.FailedSubmissions++
}
}
```

결론

BatchSubmitter 시스템은 OPinit 롤업 아키텍처에서 데이터 가용성을 보장하는 핵심 컴포넌트입니다. 주요 특징은 다음과 같습니다:

1. **비용 효율성**: Calldata 저장을 통한 가스 비용 최적화

2. **확장성**: 다양한 DA 솔루션(Celestia 등)으로 교체 가능

3. **보안성**: 권한 기반 배치 제출 및 데이터 무결성 보장

4. **결정론성**: 저장된 배치 데이터를 통한 전체 롤업 체인 재구성 가능

이러한 설계를 통해 BatchSubmitter는 롤업의 핵심 요구사항인 데이터 가용성, 비용 효율성, 보안성을 모두 만족시키는 솔루션을 제공합니다.